192472371
S.Sirivally

SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: MLA03
COURSE NAME: Reinforcement learning for Environment and Agent

---

COURSE OUTCOME COVERED

CO3: *Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)*

Assessment Tool Weightage: 30%

Dr G. A. SENTHIL

QUESTIONS: 10 **Total Marks: 50**

Annexure A

CODING CHALLENGE **Duration: 2hrs**

1.Predict the Reinforcement Learning (RL) framework by describing the agent–environment interaction, emphasizing the importance of states, actions, and rewards, and discuss how cumulative reward influences decision-making. (5 marks)

Problem Statement

Develop a simple Reinforcement Learning framework that demonstrates the interaction between an agent and its environment. The agent observes the current state, selects an action, receives a reward, and learns to maximize the cumulative reward over multiple steps.

Problem Solving Steps

1. Define the environment with three states: Start, Middle, and Goal.
2. Define the possible actions: Left and Right.
3. Initialize the agent at the Start state.
4. Select actions randomly.

5. Update the state according to the selected action.
6. Assign rewards based on the action taken.
7. Calculate the cumulative reward.
8. Stop when the Goal state is reached or the maximum number of steps is completed.

CODE

```
import random
states = ["Start", "Middle", "Goal"]
actions = ["Left", "Right"]
state = "Start"
total_reward = 0
print("Simple RL Framework\n")
for step in range(5):
    action = random.choice(actions)
    if state == "Start" and action == "Right":
        state = "Middle"
        reward = 5
    elif state == "Middle" and action == "Right":
        state = "Goal"
        reward = 10
    else:
        reward = -1
    total_reward += reward
    print(f"Step {step+1}")
    print("Action :", action)
    print("State  :", state)
    print("Reward :", reward)
    print()
```

```
    if state == "Goal":
        break
print("Total Reward =", total_reward)
```

Output

```
print("Total Reward =", total_reward)
```

```
Simple RL Framework

Step 1
Action : Left
State  : Start
Reward : -1

Step 2
Action : Left
State  : Start
Reward : -1

Step 3
Action : Right
State  : Middle
Reward : 5

Step 4
Action : Left
State  : Middle
Reward : -1

Step 5
Action : Left
State  : Middle
Reward : -1

Total Reward = 1
```

Result

- The RL framework successfully demonstrates the agent–environment interaction.
- The agent learns through rewards received from the environment.
- The cumulative reward indicates the quality of the decisions made by the agent.

192472371
S.Sirivally

2. Estimate how decision-making problems can be modeled as a Markov Decision Process (MDP), detailing the components of action space, transition probability, reward function, and discount factor. (5 marks)

Problem Statement

Develop a simple Markov Decision Process (MDP) to model sequential decision-making using states, transition probabilities, reward functions, and discount factors.

Problem Solving Steps

1. Define the states (A, B, Goal).
2. Define state transitions.
3. Assign rewards for each state.
4. Move from one state to another.
5. Display rewards after every transition.
6. Stop when the Goal state is reached.

CODE

Python

```
# Coding Challenge 2

# Simple Markov Decision Process

states = ["A", "B", "Goal"]

transition = {

   "A": "B",

   "B": "Goal"

}

reward = {

   "A": 5,

   "B": 10,

   "Goal": 20

}
```

```
state = "A"

print("Markov Decision Process\n")

while state != "Goal":

    print("Current State :", state)

    next_state = transition[state]

    print("Next State    :", next_state)

    print("Reward        :", reward[next_state])

    print("-----------------------")

    state = next_state

print("Goal Reached")
```

Output

```
        print("Reward        :", reward[next_state])

        print("-------------------------")

        state = next_state

    print("Goal Reached")
```

```
Markov Decision Process

Current State : A
Next State    : B
Reward        : 10
-----------------------
Current State : B
Next State    : Goal
Reward        : 20
-----------------------
Goal Reached
```

Result

- The MDP successfully models sequential decision-making.
- The agent reaches the Goal state through defined state transitions.
- Rewards help evaluate the quality of each transition.

192472371
S.Sirivally

3. Discuss the role of policy and value functions in RL, and examine how state-value and action-value functions contribute to evaluating long-term benefits of actions using the Bellman Equation. (5 marks)

Problem Statement

Implement the Bellman Equation to estimate the value of the current state using the immediate reward and discounted future rewards.

Problem Solving Steps

1. Define the immediate reward.
2. Define the future state value.
3. Select the discount factor (γ).
4. Apply the Bellman Equation.
5. Calculate the current state value.
6. Display the calculated value.

CODE

Python

```
# Coding Challenge 3

# Bellman Equation

gamma = 0.9

reward = 10

future_value = 20

current_value = reward + gamma * future_value

print("Bellman Equation")

print("Reward =", reward)

print("Future Value =", future_value)

print("Discount Factor =", gamma)

print("Current State Value =", current_value)
```

Output

```
print("Current State Value =", current_value)
```

```
Bellman Equation
Reward = 10
Future Value = 20
Discount Factor = 0.9
Current State Value = 28.0
```

Result

- The Bellman Equation successfully estimates the value of the current state.
- Future rewards are discounted using the discount factor.
- Higher future rewards increase the current state value.

4. Justify the exploration–exploitation trade-off in RL and compare strategies such as ε-greedy and softmax approaches in balancing learning efficiency. (5 marks)

Problem Statement

Implement the ε-greedy strategy to balance exploration and exploitation while selecting actions in a Reinforcement Learning environment.

Problem Solving Steps

1. Initialize Q-values.
2. Define epsilon (ε).
3. Generate a random number.
4. If random number < ε → Exploration.
5. Otherwise → Exploitation.
6. Repeat for multiple iterations.
7. Display selected actions.

CODE

Python

192472371
S.Sirivally

```
# Coding Challenge 4

# Epsilon Greedy

import random

epsilon = 0.2

Q = [5, 8, 2, 10]

print("Epsilon Greedy\n")

for i in range(10):
    if random.random() < epsilon:
        action = random.randint(0, 3)
        strategy = "Exploration"
    else:
        action = Q.index(max(Q))
        strategy = "Exploitation"
    print("Step", i+1)
    print("Action :", action)
    print("Strategy :", strategy)
    print()
```

Output

```
Epsilon Greedy

Step 1
Action : 3
Strategy : Exploitation

Step 2
Action : 3
Strategy : Exploitation

Step 3
Action : 3
Strategy : Exploitation

Step 4
Action : 3
Strategy : Exploitation

Step 5
Action : 3
Strategy : Exploitation

Step 6
Action : 3
Strategy : Exploitation

Step 7
Action : 3
Strategy : Exploitation

Step 8
Action : 3
Strategy : Exploitation

Step 9
Action : 3
Strategy : Exploitation

Step 10
Action : 2
Strategy : Exploration
```

Result

- Exploration allows the agent to discover new actions.
- Exploitation selects the best-known action.
- ε-greedy provides a balance between learning and performance.

192472371
S.Sirivally

5. Illustrate with an example how Q-learning updates the action-value function during training, and demonstrate its convergence behavior. (5 marks)

Problem Statement

Implement the Q-learning update equation to update the action-value function and demonstrate how the Q-value converges toward the optimal value.

Problem Solving Steps

1. Initialize the Q-value.
2. Define the learning rate (α).
3. Define the discount factor (γ).
4. Observe the reward.
5. Estimate the future Q-value.
6. Apply the Q-learning update equation.
7. Display the updated Q-value.

CODE

Python

```
# Coding Challenge 5

# Q-Learning Update

alpha = 0.5

gamma = 0.9

Q = 10

reward = 5

future_Q = 20

new_Q = Q + alpha * (reward + gamma * future_Q - Q)

print("Q-Learning")

print("Old Q Value :", Q)

print("Reward :", reward)

print("Future Q :", future_Q)
```

```
print("Updated Q :", round(new_Q,2))
```

Output

```
Q-Learning
Old Q Value : 10
Reward : 5
Future Q : 20
Updated Q : 16.5
```

Result

- The Q-value is successfully updated using the Q-learning equation.
- Updated Q-values improve action selection.
- Repeated updates lead to convergence toward the optimal policy.

Code of Conduct:

I __S.Sirivally / 192472371__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Sirivally.S

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately